Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Theory, Edge Cases & Code Examples

- ✓ The Problem with Standard Allocation
- ✓ Memory Pool Design
- ✓ Free List Management
- ✓ Alignment Considerations
- ✓ Alignment Issues

→ Swipe to learn →

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Theory

📌 1. The Problem with Standard Allocation

Standard Heap Allocation Issues:

```
// ❌ Problem: Each allocation has overhead
for (int i = 0; i < 1000000; ++i) {
    Node* node = new Node();  // Calls malloc, slow!
    // - System call overhead
    // - Fragmentation
    // - Unpredictable latency
    // - Poor cache locality
}
```

Performance Impact:

- `malloc` / `new` : ~100-500ns per allocation
- Memory pool: ~5-20ns per allocation (10-50x faster!)
- External fragmentation wastes memory
- Poor cache performance (allocated objects scattered)

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Theory

📌 2. Memory Pool Design

Core Concept:

Pre-allocate a large contiguous block and manage it ourselves:

```
// ✅ Memory pool approach
class MemoryPool {
    char* memory_block_;      // Pre-allocated chunk
    void* free_list_;         // Linked list of free blocks
    size_t block_size_;       // Fixed size per object
    size_t block_count_;      // Total blocks
};
```

Key Advantages:

1. **O(1) Allocation/Deallocation**: Just pop/push from free list
2. **No Fragmentation**: Fixed-size blocks
3. **Cache Locality**: Objects allocated near each other
4. **Predictable Performance**: No system calls after initialization

---

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Theory

📌 3. Free List Management

Intrusive Free List:

Store the free list inside the free blocks themselves:

```
// Each free block stores a pointer to the next free block
struct FreeBlock {
    FreeBlock* next;
};

// Memory layout:
// [Used][Free*→][Free*→][Used][Free*→nullptr]
//         ↓       ↓             ↓
//       Next    Next         Last
```

Why This Works:

- Free blocks are unused, so we can store pointers in them
- Zero overhead when blocks are allocated
- Simple linked list operations

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Theory

📌 4. Alignment Considerations

Memory Alignment Rules:

```
// Must align to object's alignment requirements
alignof(int) == 4      // int must be 4-byte aligned
alignof(double) == 8   // double must be 8-byte aligned
alignof(std::max_align_t) == 16  // Maximum alignment

// Misaligned access:
char* ptr = new char[100];
int* p = reinterpret_cast<int*>(ptr + 1);  // ❌ Misaligned!
*p = 42;  // Undefined behavior or crash on some platforms
```

Solution:

```
// Ensure block_size is properly aligned
size_t aligned_size = (size + alignof(T) - 1) & ~(alignof(T) - 1);
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Edge Cases

📌 1. Alignment Issues

Problem:

```
struct Misaligned {
    char c;
    double d;  // Needs 8-byte alignment
};

MemoryPool<Misaligned> pool(100);
auto* obj = pool.allocate();
obj->d = 3.14;  // May crash if misaligned!
```

Solution:

Use `std::aligned_alloc` and union for proper alignment.

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Edge Cases

📌 2. Double-Free

Problem:

```
T* ptr = pool.construct(42);
pool.destroy(ptr);
pool.destroy(ptr);  // ❌ Double-free! Corrupts free list
```

Detection:

```
void deallocate(T* ptr) {
    // Check if already in free list (expensive, debug only)
    #ifdef DEBUG
    for (Block* b = free_list_; b; b = b->next) {
        if (b == reinterpret_cast<Block*>(ptr)) {
            throw std::logic_error("Double-free detected");
        }
    }
    #endif
    // ... normal deallocation
}
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Edge Cases

📌 3. Pool Exhaustion

Problem:

```
MemoryPool<int> pool(10);
for (int i = 0; i < 100; ++i) {
    pool.allocate();  // ❌ Throws after 10 allocations
}
```

Handling:

```
T* safe_allocate(MemoryPool<T>& pool) {
    try {
        return pool.allocate();
    } catch (const std::bad_alloc&) {
        // Handle exhaustion: fallback to heap, grow pool, etc.
        return new T();
    }
}
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Edge Cases

📌 4. Destructor Not Called

Problem:

```
{
    MemoryPool<std::string> pool(10);
    auto* str = pool.allocate();  // Memory allocated
    new (str) std::string("Hello");  // Object constructed
}  // ❌ Pool destroyed, but string destructor never called!
   // Memory leak in std::string's internal buffer
```

Solution:

Always use `construct` / `destroy` pair:

```
{
    MemoryPool<std::string> pool(10);
    auto* str = pool.construct("Hello");  // ✅
    // ... use str ...
    pool.destroy(str);  // ✅ Calls destructor
}
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Edge Cases

📌 5. Cache Line False Sharing

Problem:

```
struct Counter {
    std::atomic<int> value;  // Only 4 bytes
};

// Multiple threads accessing adjacent counters
MemoryPool<Counter> pool(100);
auto* c1 = pool.construct(0);  // Thread 1
auto* c2 = pool.construct(0);  // Thread 2 (likely same cache line)

// False sharing: c1 and c2 on same cache line
// Thread 1 and 2 thrash cache, huge performance hit!
```

Solution:

Pad to cache line size:

```
struct alignas(64) Counter {  // 64 = typical cache line size
    std::atomic<int> value;
    char padding[60];  // Force different cache lines
};
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 1. Basic Usage (1/2)

This example demonstrates the fundamental operations of a memory pool: **allocating, constructing, using, and destroying objects**. The key advantage shown here is **O(1) constant-time allocation** - the pool simply pops a block from the free list, which is dramatically faster than `malloc` or `new`.

What this code does:
- Creates a memory pool with capacity for 1000 Node objects
- Uses `construct()` to both allocate memory AND call the constructor (placement new)
- Builds a small linked list of nodes
- Uses `destroy()` to properly call destructors AND return memory to the pool
- Shows that after allocating and destroying 3 objects, all 1000 slots are available again

Key concepts demonstrated:
- **Placement new**: `construct()` uses placement new internally to call the constructor on pre-allocated memory
- **Explicit destruction**: Unlike `delete`, we must explicitly call the destructor before returning memory
- **Reusability**: Destroyed blocks immediately return to the free list for reuse
- **No system calls**: After initialization, all operations are pure pointer arithmetic

Why this matters: In high-performance scenarios (game engines, real-time systems, network servers), avoiding `malloc` overhead can improve performance by 10-50x for small object allocations.

Output:

```
#include <iostream>

struct Node {
    int data;
    Node* next;

    Node(int d) : data(d), next(nullptr) {}
};

int main() {
    MemoryPool<Node> pool(1000);

// ...continues on next slide
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 1. Basic Usage (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    // Allocate and construct
    Node* n1 = pool.construct(10);
    Node* n2 = pool.construct(20);
    Node* n3 = pool.construct(30);

    // Use nodes
    n1->next = n2;
    n2->next = n3;

    // Clean up
    pool.destroy(n1);
    pool.destroy(n2);
    pool.destroy(n3);

    std::cout << "Pool capacity: " << pool.capacity() << '\n';
    std::cout << "Available: " << pool.available() << '\n';
}
```

```
['Pool capacity: 1000\nAvailable: 1000']
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 2. Linked List with Pool (1/2)

This example demonstrates a real-world use case: **building a custom container (linked list) on top of a memory pool**. This is exactly how high-performance libraries like game engines manage dynamic data structures. The key insight is that **all allocations come from the pool, eliminating heap fragmentation and ensuring predictable performance**.

What this code does:
- Implements a `PooledLinkedList` class that owns its memory pool
- Each node is allocated from the pool, not from the heap
- The destructor walks the list and properly destroys each node, returning memory to the pool
- `push_front` uses `pool.construct()` to allocate and initialize nodes in one step
- `pop_front` uses `pool.destroy()` to destruct and deallocate nodes

Key benefits demonstrated:
- **Zero fragmentation**: All nodes are the same size and allocated from a contiguous block
- **Memory locality**: Nodes allocated sequentially are likely adjacent in memory, improving cache performance
- **Predictable deallocation**: Destroying the list takes exactly N operations, no hidden `free()` costs
- **Compile-time sizing**: Pool capacity is fixed at construction, preventing unbounded growth

Real-world application: This pattern is used in:
- **Game engines**: Entity component systems allocate thousands of components per frame
- **Network servers**: Connection objects, packet buffers, request handlers
- **Databases**: B-tree nodes, hash table buckets, transaction logs

Performance impact: For a linked list with 10,000 nodes, pool-based allocation can be **20-50x faster** than standard `new` / `delete`, and uses **2-3x less memory** due to reduced overhead.

```
template<typename T>
class PooledLinkedList {
    struct Node {
        T data;
        Node* next;
        Node(const T& d) : data(d), next(nullptr) {}
    };

// ...continues on next slide
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 2. Linked List with Pool (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    MemoryPool<Node> pool_;
    Node* head_;

public:
    PooledLinkedList(size_t pool_size)
        : pool_(pool_size), head_(nullptr) {}

    ~PooledLinkedList() {
        while (head_) {
            Node* temp = head_;
            head_ = head_->next;
            pool_.destroy(temp);
        }
    }

    void push_front(const T& value) {
        Node* new_node = pool_.construct(value);
        new_node->next = head_;
        head_ = new_node;
    }

    void pop_front() {
        if (head_) {
            Node* temp = head_;
            head_ = head_->next;
            pool_.destroy(temp);
        }
    }
};

int main() {
    PooledLinkedList<int> list(100);

    for (int i = 0; i < 10; ++i) {
        list.push_front(i);
    }
}
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 3. Performance Comparison (1/3)

This example provides **empirical proof** of the memory pool's performance advantage through a head-to-head benchmark. The typical result shows **15x speedup** (45ms vs 3ms for 100,000 allocations), demonstrating why memory pools are essential for high-performance C++ applications.

What this code measures:
- **Standard allocation**: Uses `new` and `delete` for each of 100,000 Data objects
- **Pool allocation**: Uses `pool.construct()` and `pool.destroy()` for the same operations
- Both approaches allocate objects into a vector and then deallocate them all
- High-resolution timer measures the total time for allocation + deallocation

Why the pool is 15x faster:
1. **System call overhead**: Each `new` may invoke `malloc`, which can make system calls (brk/mmap). Pool makes NO system calls after initialization.
2. **Allocation algorithm cost**: `malloc` must search for free blocks using complex algorithms (buddy system, bins, etc.). Pool just pops from a linked list (1-2 instructions).
3. **Cache effects**: Pool allocations are sequential in memory, so CPU prefetcher can predict and load next blocks. Heap allocations are scattered, causing cache misses.
4. **TLB thrashing**: Heap allocations may span multiple memory pages, causing TLB (translation lookaside buffer) misses. Pool uses fewer pages.
5. **Bookkeeping overhead**: `malloc` maintains metadata for each allocation (size, flags, magic numbers). Pool has zero per-allocation overhead.

Real-world implications:
- **Game engine** allocating 100,000 particles per frame: 45ms → 3ms means 60 FPS becomes possible
- **Trading system** processing 1M orders/second: Reduced latency from microseconds to nanoseconds
- **Embedded system** with limited memory: Pool prevents fragmentation, ensuring consistent performance over days/weeks

Important caveat: Pool advantage decreases if:
- Objects are large (>1KB): Allocation time dominated by memory initialization
- Low allocation frequency: One-time costs matter less
- Variable object sizes: Pool design assumes fixed size

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 3. Performance Comparison (2/3)

```
#include <chrono>
#include <vector>

struct Data {
    int values[10];
};

void benchmark_standard_allocation() {
    auto start = std::chrono::high_resolution_clock::now();

    std::vector<Data*> ptrs;
    for (int i = 0; i < 100000; ++i) {
        ptrs.push_back(new Data());
    }
    for (auto* ptr : ptrs) {
        delete ptr;
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
    std::cout << "Standard: " << duration.count() << "ms\n";
}

void benchmark_pool_allocation() {
    auto start = std::chrono::high_resolution_clock::now();

    MemoryPool<Data> pool(100000);
    std::vector<Data*> ptrs;
    for (int i = 0; i < 100000; ++i) {
        ptrs.push_back(pool.construct());
    }
    for (auto* ptr : ptrs) {
        pool.destroy(ptr);
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
    std::cout << "Pool: " << duration.count() << "ms\n";
}

int main() {
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 3. Performance Comparison (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    benchmark_standard_allocation();  // ~45ms
    benchmark_pool_allocation();      // ~3ms (15x faster!)
}
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 4. Thread-Local Pool for Multithreading (1/3)

This example demonstrates **thread-safe memory allocation using thread-local pools**, eliminating lock contention entirely. Instead of using a single shared pool protected by mutexes (which causes thread serialization), each thread gets its own private pool. This achieves **perfect scalability** - adding more threads doesn't slow down allocation.

What this code does:
- Uses C++11's `thread_local` storage to give each thread its own dedicated memory pool
- Worker threads perform intensive allocations without any synchronization overhead
- Each thread allocates 10,000 objects independently and in parallel
- No locks, no contention, no cache-line bouncing between cores

Key advantages over shared pool:
- **Zero lock overhead**: Each thread owns its pool exclusively
- **No cache contention**: Thread-local pools live in each core's L1/L2 cache
- **Perfect scalability**: 8 threads = 8x throughput (linear scaling)
- **NUMA-friendly**: On multi-socket systems, pools are local to each NUMA node

Tradeoff: Memory overhead grows with thread count (each thread allocates full pool). For 8 threads with 10K capacity each = 80K total capacity vs 10K for shared pool.

Real-world usage:
- **Game engines**: Per-thread particle systems, physics objects, AI agents
- **Web servers**: Per-request memory pools (request handled by one thread)
- **Database systems**: Per-transaction allocators

Performance comparison:

```
#include <thread>
#include <vector>
#include <iostream>

struct Object {
    int data[100];  // Larger object to amplify allocation cost
};

// ...continues on next slide
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 4. Thread-Local Pool for Multithreading (2/3)

[Code continues across slides]

```
// ...continued from previous slide
// Each thread gets its own pool (no sharing!)
thread_local MemoryPool<Object> thread_pool(10000);

void worker_thread(int thread_id) {
    std::vector<Object*> objects;

    // Allocate 10,000 objects - NO LOCKS NEEDED!
    for (int i = 0; i < 10000; ++i) {
        objects.push_back(thread_pool.construct());
    }

    // Do work with objects...

    // Clean up
    for (auto* obj : objects) {
        thread_pool.destroy(obj);
    }

    std::cout << "Thread " << thread_id << " completed\n";
}

int main() {
    const int num_threads = 8;
    std::vector<std::thread> threads;

    // Launch threads - each uses its own pool
    for (int i = 0; i < num_threads; ++i) {
        threads.emplace_back(worker_thread, i);
    }

    // Wait for completion
    for (auto& t : threads) {
        t.join();
    }

    std::cout << "All threads completed with zero lock contention!\n";
}
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 4. Thread-Local Pool for Multithreading (3/3)

```
['Shared pool with mutex:     ~500ms for 8 threads (serialization)\nThread-local pools:        ~70ms for 8 threads (tr
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 5. RAII Pool Wrapper for Exception Safety (1/2)

This example demonstrates **automatic cleanup using RAII** to prevent memory leaks when exceptions occur. The `PoolPtr` smart pointer wrapper ensures objects are always destroyed and returned to the pool, even if exceptions are thrown during processing.

What this code does:
- Implements a `PoolPtr<T>` smart pointer that wraps pool-allocated objects
- Automatically calls `pool.destroy()` in its destructor (RAII pattern)
- Works with C++11 move semantics to transfer ownership efficiently
- Prevents leaks even when exceptions occur mid-processing

Key safety guarantees:
- **Exception-safe**: Object destroyed even if exception thrown
- **No double-free**: Move semantics transfer ownership, original pointer becomes null
- **RAII compliance**: Follows "Resource Acquisition Is Initialization" pattern
- **Zero overhead**: No runtime cost compared to manual destroy() calls

Why this matters:
Without RAII, exceptions cause leaks:

With RAII, automatic cleanup:

Real-world application:
- **Transaction processing**: Rollback on exception
- **Game loop**: Ensure resources freed each frame, even on crash
- **Request handling**: Prevent resource leaks in long-running servers

Output:

The pool is fully recovered even though an exception was thrown, demonstrating perfect exception safety.

```
// ❌ UNSAFE - leaks if process() throws
Object* obj = pool.construct();
process(obj);  // May throw!
pool.destroy(obj);  // Never reached if exception thrown
```

Fixed-Size Memory Pool Allocator - High-Performance Memory Management

Code Examples

📌 5. RAII Pool Wrapper for Exception Safety (2/2)

```
['// ✅ SAFE - always cleaned up\nPoolPtr<Object> obj(pool);\nprocess(*obj);  // If throws, destructor still runs\n// A
```

What You Learned

Key Takeaways

- ✓ The Problem with Standard Allocation
- ✓ Memory Pool Design
- ✓ Free List Management
- ✓ Alignment Considerations

→ Swipe to learn →